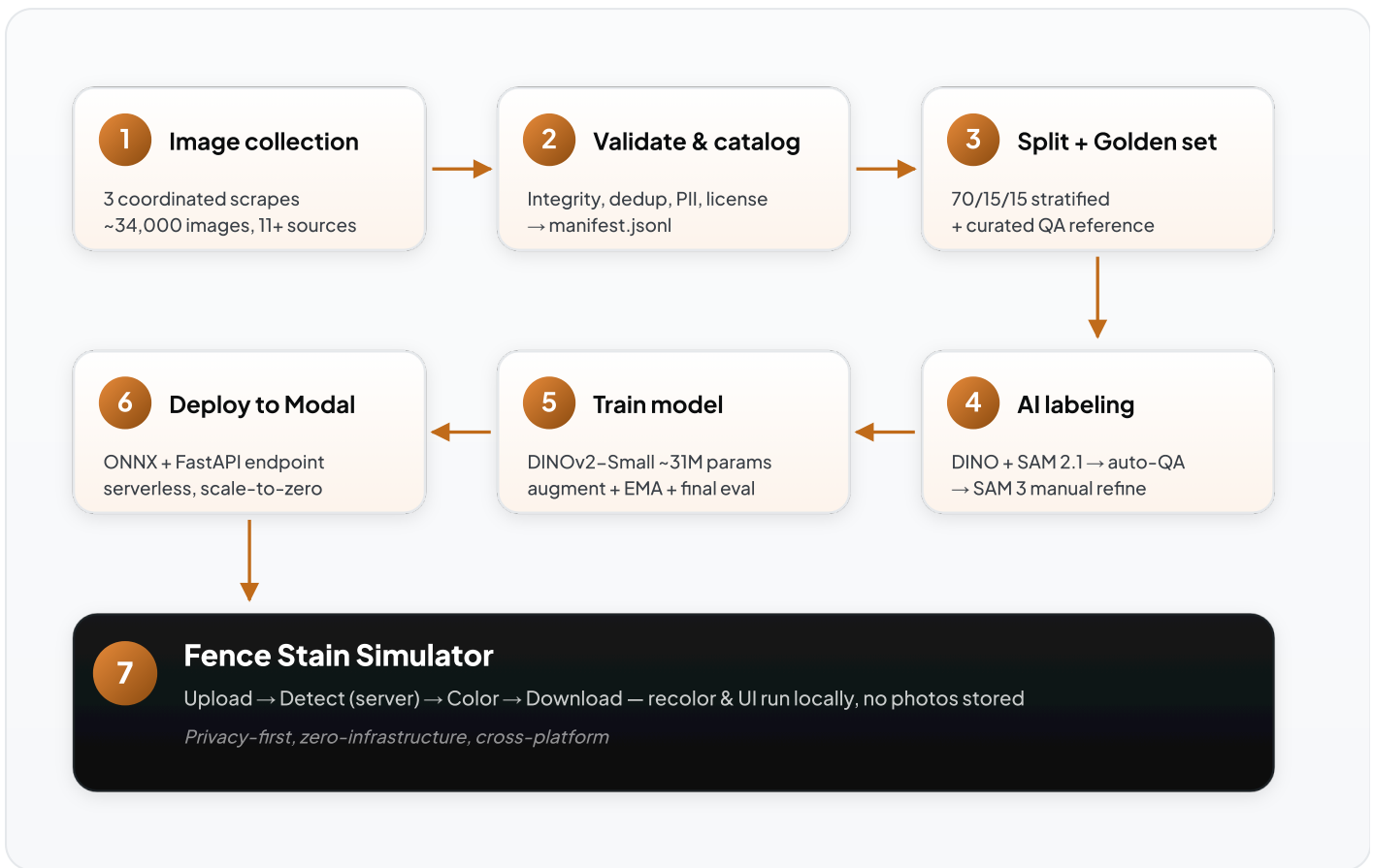


A client-friendly walk-through of how the Fence Stain Simulator fence-visualizer goes from raw web photos all the way to the in-browser experience your customers click on. No deep ML jargon — just the moving parts and what each one does.

What Fence Stain Simulator is

Fence Stain Simulator is a browser-based tool that lets a homeowner upload a photo of their fence, automatically detects the fence boards using AI, and previews any stain color on it instantly. The AI detection step runs on a tiny **serverless backend** that processes the photo in memory and discards it the moment the response is sent — no logging, no persistence. Everything else — color picking, recoloring, downloading — happens **locally in the browser**, so changing colors or opacity is instant.

The pipeline at a glance



Step 1 — Image collection (scraping)

To teach the model what a fence looks like (and what doesn't), we gather a broad, real-world photo library from many sources.

Three coordinated scrapes:

Scrape	Goal	Target volume	What it gathers
Positives	Fences we <i>can</i> stain	~17,000 images	Cedar, wood, picket, privacy, shadowbox fences in every angle, season, and condition
Hard positives	Tricky-but-valid fences	~5,000 images (top-up)	Heavy plant occlusion, snow, broken boards, half-stained, dramatic lighting, unusual angles
Negatives	Look-alikes we should <i>not</i> stain	~12,000 images	Vinyl/chain-link/iron fences, brick walls, wood siding, decks, pergolas, hedges, empty yards
Combined	Full training pool	~34,000 images	Balanced across ~30 fence styles and ~20 look-alike categories

Where the photos come from: Pexels, Unsplash, Pixabay, Wikimedia, plus respectful crawls of Google, Bing, Pinterest, Houzz, and ~30 licensed fence-company galleries (~200 unique search queries across the three scrapes). The whole scraper is built for production use — rate limiting per source, retry-on-failure, duplicate detection, content filtering, and Google Vision verification on positives to catch off-topic results before they enter the dataset. Source licensing is tracked per image for clean commercial use.

Output: `data_scraped/` (positives) and `data_scraped_neg/` (negatives), each with the images plus a metadata file recording where each photo came from, its license, and quality details.

Step 2 — Validate, clean, and catalog

Raw scraped data always has some noise, and a few compliance checks need to happen before any image enters the training pool. This stage runs a single audited pass that:

- **Verifies every image** opens correctly (catches corrupt or truncated files)
- **Removes duplicates** within and across the positive and negative sets (SHA-256 byte match + visual-hash near-duplicate detection)
- **Removes orphans** (files without metadata, or metadata without files)
- **Scans for PII** — a Google Vision OCR pass catches readable text on signs, vehicles, or house numbers so we can review or drop sensitive photos

- **Audits every image's license** per source — the resulting report is the legal paper trail for clean commercial use
- **Standardizes** image orientation, EXIF rotation, and color profile so the model sees consistent input regardless of how the photo was uploaded
- **Drops near-empty fence samples** (images where the fence is fewer than ~100 pixels — so occluded the image carries no learning signal)
- **Tags each image** with a subcategory — e.g., *cedar*, *weather*, *occlusion*, *multi-structure*, *brick-wall*, *chain-link* — so the training pool is balanced across real-world situations
- **Produces a single catalog file** (`dataset/manifest.jsonl`) that becomes the single source of truth for the rest of the pipeline

Output: a clean, fully-cataloged dataset with integrity + license audit reports kept alongside it for reproducibility.

Step 3 — Splitting into train / validate / test (+ Golden set)

The catalog is divided into three slices the AI never confuses:

- **Training set (~70%)** — what the model learns from
- **Validation set (~15%)** — used during training to check progress
- **Test set (~15%)** — held back to honestly measure the final model

The split is **stratified** — every subcategory (cedar fence, vinyl fence, empty yard, occlusion, etc.) is proportionally represented in all three slices, so the model never gets surprised by a category it hardly saw. We also use visual-similarity grouping to make sure near-duplicate photos always end up in the same slice (preventing the model from "cheating" by seeing a near-copy of a test image during training).

On top of the three splits we curate a small **Golden Set** — a hand-picked reference batch of photos covering the trickiest real-world conditions (deep occlusion, harsh lighting, similar-looking distractors, multi-fence scenes). The golden set is run after every model update to spot regressions that matter in practice but might hide inside aggregate numbers.

Once the labeling step (next) finishes, the produced pixel masks are attached to each slice — every training image is paired with its mask so the model can learn from both at the same time.

Output: `dataset/splits/{train,val,test}.jsonl` plus matching mask files, an audit record for reproducibility, and `dataset/golden_set/` for ongoing QA.

Step 4 — AI-assisted labeling

For each training image, we need to know *exactly which pixels are fence*. Hand-drawing those masks for tens of thousands of images would take months. Instead we use a three-stage pipeline that combines AI detection, automated quality control, and targeted human review:

Stage 1 — AI detection + mask generation

1. **Grounding DINO** — an open-vocabulary detector that finds objects matching text prompts ("wooden fence", "cedar fence", "wood plank fence", plus a longer list of variations).
2. **SAM 2.1** (Segment Anything) — converts each detection box into a pixel-accurate mask.

Stage 2 — Automatic quality gate

1. **Auto-accept** high-confidence positives — masks that clear the detection-confidence + mask-coverage thresholds are approved directly, with no human time spent.
2. **Auto-review** uncertain or low-confidence cases — anything below the threshold is automatically flagged for human inspection, never silently accepted or rejected.

Stage 3 — Targeted manual refinement

1. **SAM 3** is used by a human reviewer to refine only the flagged cases — tricky occlusion, gates, edge boundaries, or images where the auto pipeline disagreed with itself. The reviewer only sees images that actually need attention, so manual time scales with edge-case volume, not with dataset size.

We use a **3-class schema** so the AI can also learn what *not* to stain:

- **fence_wood** — the target (will be re-stained in the app)
- **not_target** — wood siding, deck railings, vinyl fences, brick walls, furniture, etc. (looks similar, but we don't want to stain it)
- **background** — everything else

The **not_target** class is the key trick: it gives the AI an explicit category for "wood-like thing that isn't a wooden fence", which dramatically cuts down false positives.

Output: a pixel mask for every catalog image, plus an audit trail showing which masks were auto-accepted vs human-refined.

Step 5 — Train the AI model

We train a compact segmentation model designed from day one to fit in a browser:

- **DINOv2-Small backbone** — a strong, open vision model from Meta
- **Multi-scale decoder + Mask2Former head** — translates the backbone's features into a fence mask
- **Refinement module** — sharpens the mask's edges and removes obvious mistakes
- **Total size:** ~31M parameters — small enough to ship to a browser

Augmentation

Every training image is randomly perturbed so the model effectively sees thousands of variations of the same photo and learns to generalize:

- **Spatial:** random crops, horizontal flips, multi-scale resizing (75–125% of the base resolution), and CutMix — pasting a patch from one image into another so the model learns to handle blended scenes
- **Photometric:** color, brightness, contrast, and sharpness jitter so the model is robust to lighting, weather, and camera differences
- **Balanced sampling:** during each training epoch we re-weight the draw-probability of rare subcategories (e.g., damaged fences, snowy fences) so the model doesn't ignore the long tail of edge cases

Training loop

The model is updated on the training split, validated each epoch on the validation split, and an exponential-moving-average copy of the weights is kept in parallel for smoother, more stable predictions. When training finishes, the best validation checkpoint is selected and evaluated once on the held-back **test split** — with test-time augmentation (averaging predictions over multiple scales and a horizontal flip) and the same post-processing the browser will apply. That run produces the final honest evaluation report and a side-by-side preview gallery.

Output: a trained model checkpoint, the test-eval report, and a sample-image preview gallery showing model behavior on real photos.

Step 6 — Convert and deploy to Modal

The trained model is exported to **ONNX format** — an open, framework-neutral standard that any inference runtime can load. The ONNX file plus a small FastAPI handler are bundled into a **Modal** container image that exposes a single `/detect` HTTP endpoint.

Why server-side instead of in-browser:

- The page loads in **under a second** — no large model file download
- Inference runs on managed CPU (~500 ms per image), so the experience is the same on a phone, a laptop, or an older browser
- The model can be retrained and redeployed independently of the website — no client update needed
- Modal **scales to zero when idle** (no idle cost) and pre-warms on page load so the first detection click is fast

We also generate a small sidecar file (`fence_model_dinov2.json`) that records exactly how the model expects its input, used during server-side preprocessing for parity with training.

Output: a deployed HTTPS endpoint that accepts a JPEG upload and returns a compact PNG mask (~30-100 KB), plus the bundled ONNX in the container image.

Step 7 — The Fence Stain Simulator web visualizer

The browser app is a single lightweight HTML page (`fence-staining-visualizer/index2.html`) that talks to the Modal endpoint from Step 6 only when the user clicks *Detect*. Everything else — color picking, blending, downloading — happens locally in the browser.

User flow:

1. **Upload** — drag-and-drop or pick a fence photo (JPG / PNG / WebP, up to 10 MB)
2. **Detect** — the browser sends a downsized copy to the AI service, which returns a fence mask in 1–3 seconds
3. **Color** — pick from 20 curated stain colors (Cedar, Walnut, Charcoal, Mahogany, Pine, etc.), adjust opacity, choose a blend mode (Multiply for realistic, Overlay for vivid, Screen for highlights) — fully local, instant
4. **Download** — save the recolored preview as a PNG

Behind the user-facing screen, the app also:

- Downsizes the photo to ≤ 1024 px JPEG before upload (typically 150–400 KB), so the network transfer is tiny even on mobile data
- Pings the AI service on page load so its container is already warm when the user clicks *Detect*
- Decodes the PNG mask the server returns and uses the model's **confidence level as the blending alpha** — stained areas have natural anti-aliased edges instead of a chunky cut-out look
- Runs a **connected-component cleanup** that removes small, isolated false-positive blobs (dirt patches, far-away objects)
- Runs a **vegetation filter** that strips out green-dominant pixels (leaves and bushes growing in front of the fence) so they don't get stained
- Erodes the mask edge by one pixel to suppress slight color bleed onto the grass or sky

Recoloring, color changes, opacity tweaks, blend-mode switching, and the final download all run **client-side** — the server is only contacted for the one AI detection step.

Privacy and deployment

- **Privacy-conscious data handling:** the photo is sent to the AI service **only** when the user clicks *Detect*, transmitted over HTTPS, processed entirely in memory, and discarded the instant the response is sent. No logging, no persistence, no analytics. Color picking, recoloring, and downloading all happen locally and never leave the device.
 - **Lightweight frontend:** the web page is a single static HTML file — no large model download, fast first paint on any device. Hosted on any CDN or static host.
 - **Auto-scaling backend:** the AI runs on a managed serverless container that scales to zero when nobody is using it (no idle cost) and warms up on page load so the user's first detection click is fast.
 - **Cross-platform:** works on Chrome, Edge, Firefox, and Safari on desktop and mobile.
-

Project structure (for reference)

data_scraper/	Image collection engine (multiple sources)
data_scraped/	Raw positive images + metadata
data_scraped_neg/	Raw negative images + metadata
annotation/	AI-assisted labeling pipeline (Grounding DINO + SAM)
configs/	Scraper configs + annotation schema + training configs
dataset/	Cleaned catalog + train/val/test splits + masks
tools/	Utility scripts (mask building, ONNX export, etc.)
training/	Model architecture, losses, metrics, optimizer
prepare_dataset.py	Step 2 – clean & catalog
split_dataset.py	Step 3 – stratified split
train_web_deployable.py	Step 5 – train the deployable model
modal_inference/	Step 6 – serverless AI backend (Modal + FastAPI + ONNX)
fence-staining-visualizer/	Step 7 – the Fence Stain Simulator web frontend (HTML, calls Modal)

Fence Stain Simulator — instant AI fence color previews, in any browser.